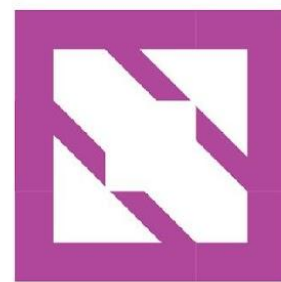




KubeCon



CloudNativeCon

Europe 2025





KubeCon



CloudNativeCon

Europe 2025

More Nodes, More Problems: Solving Multi-Host GPU/TPU Scheduling With Dynamic Resource Allocation

John Belamaric (Google)

Morten Jæger Torkildsen (Google)



Who are we?



John Belamaric
Sr Staff Software Engineer
Google



Morten Jæger Torkildsen
Sr Software Engineer
Google

AI/ML workloads can require a large number of accelerators.

As the number of accelerators and nodes grow, so does the complexity of scheduling and the chance that we might encounter failures.

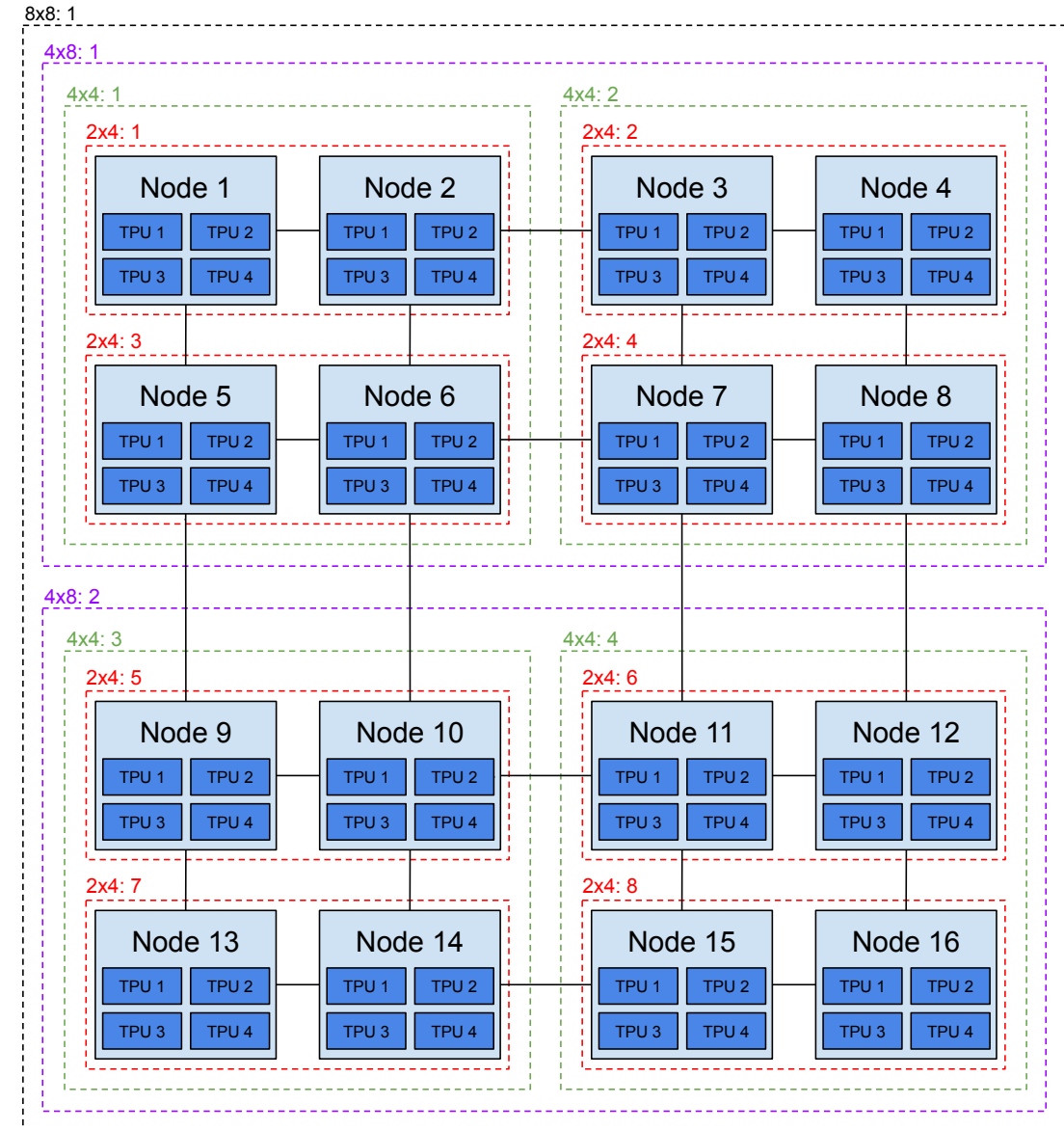
More Nodes $\Rightarrow$ More Failure Points $\Rightarrow$ More Failures

The Multi-host Challenge

- Small scale example, with TPUs:
 - 16 nodes, 4 TPUs per node $\Rightarrow$ 64 TPUs
 - Several training jobs, 16 TPUs per job
- We want:
 - Compact placement: Pods schedule on close-together nodes
 - Concurrent job scheduling without race conditions
 - Automatic handling of failures

Multi-host TPU topology

- Groups of TPUs are connected with a high-speed ICI, in a specific topology.
- Only certain groups of nodes works:
 - 1 8x8 (64) TPU slice
 - 2 4x8 (32) TPU slices
 - 4 4x4 (16) TPU slices
 - 8 2x4 (8) TPU slices
 - 16 2x2 (4) TPU slices
- (others are possible, but this keep things manageable for the example)



- Label the nodes in each slice
 - Overlapping slices possible, but risks race conditions
- Job sets the node selector on its pods
- Each pod asks for 4 TPUs

Less than ideal...

- Must choose between:
 - Static partitioning of slices; can lead to poor utilization
 - Overlapping slices; can lead to scheduling race conditions
- Must coordinate on picking node selectors
- Node failures require updating the selectors in the pod spec
- Lacks expected Kubernetes flexibility and self-healing

Let's try it out



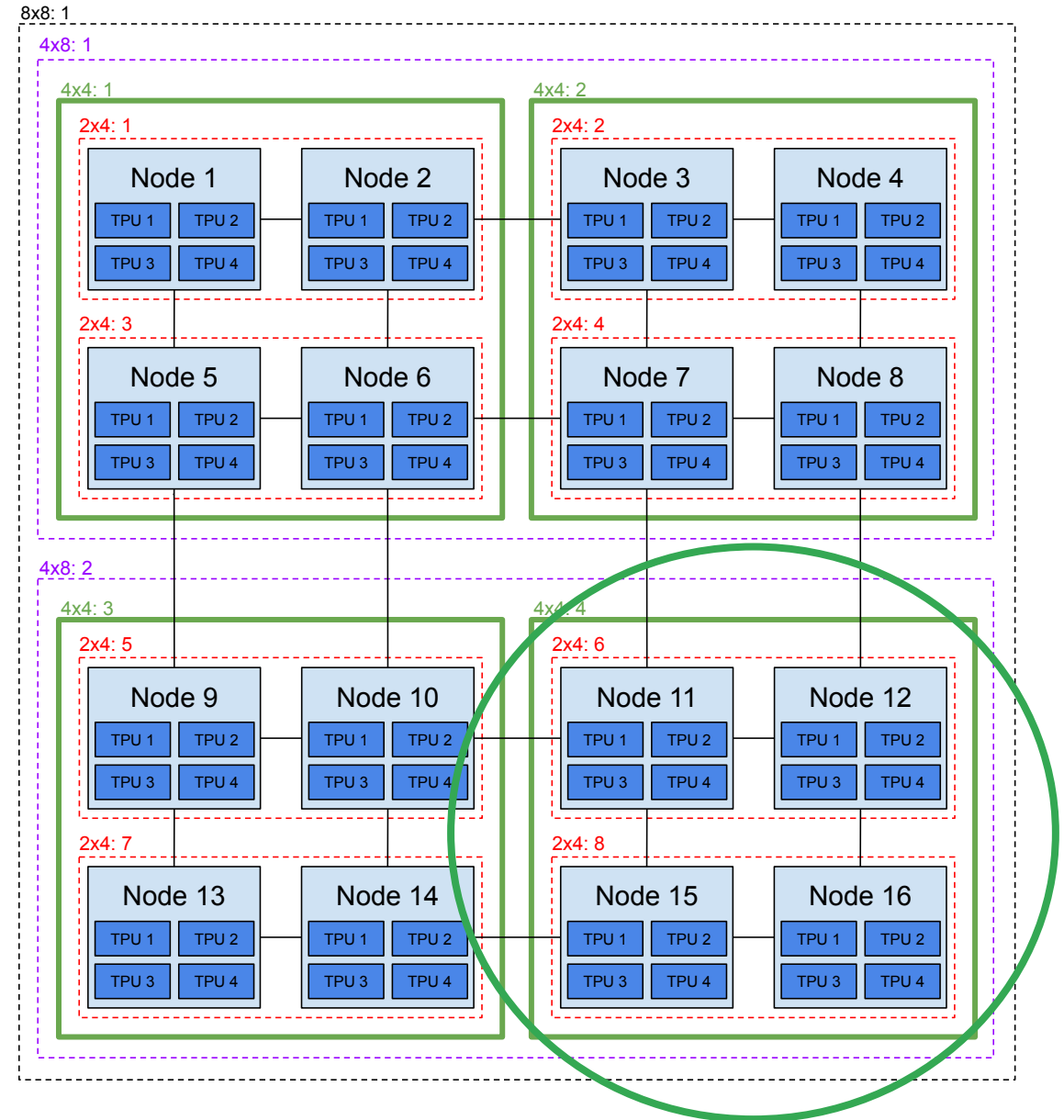
KubeCon



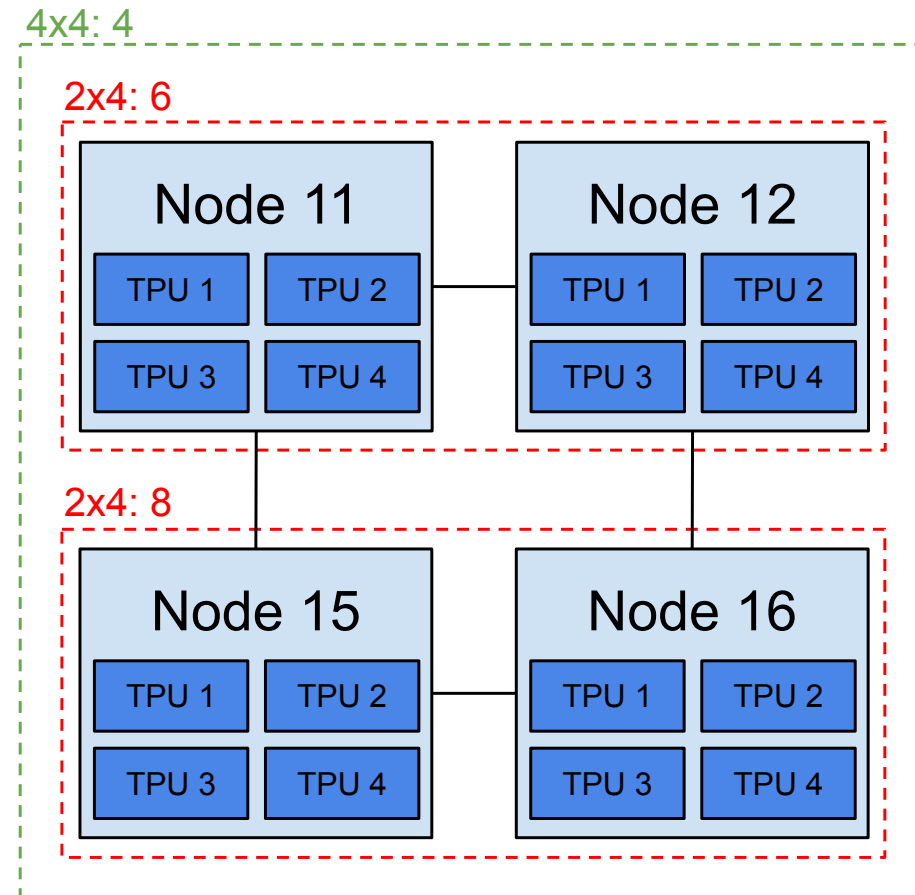
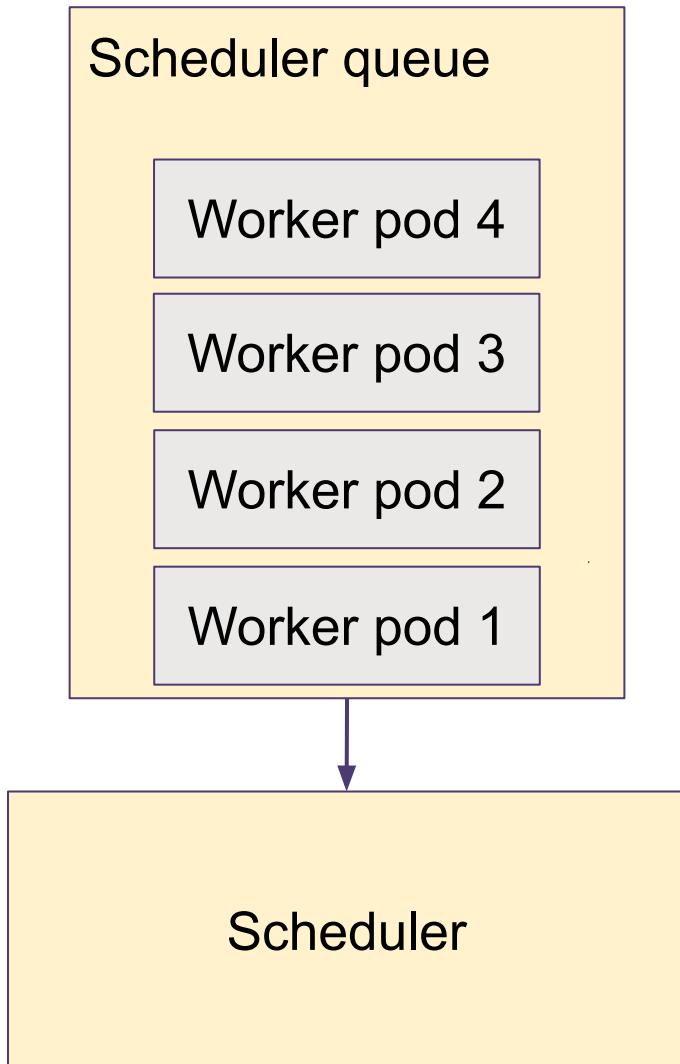
CloudNativeCon

Europe 2025

- 16 TPU job: 4x4 slice
- Manually choose a slice
 - (11, 12, 15, 16)
- Set the correct node selector



Multi-host TPUs without DRA



Multi-host TPUs without DRA

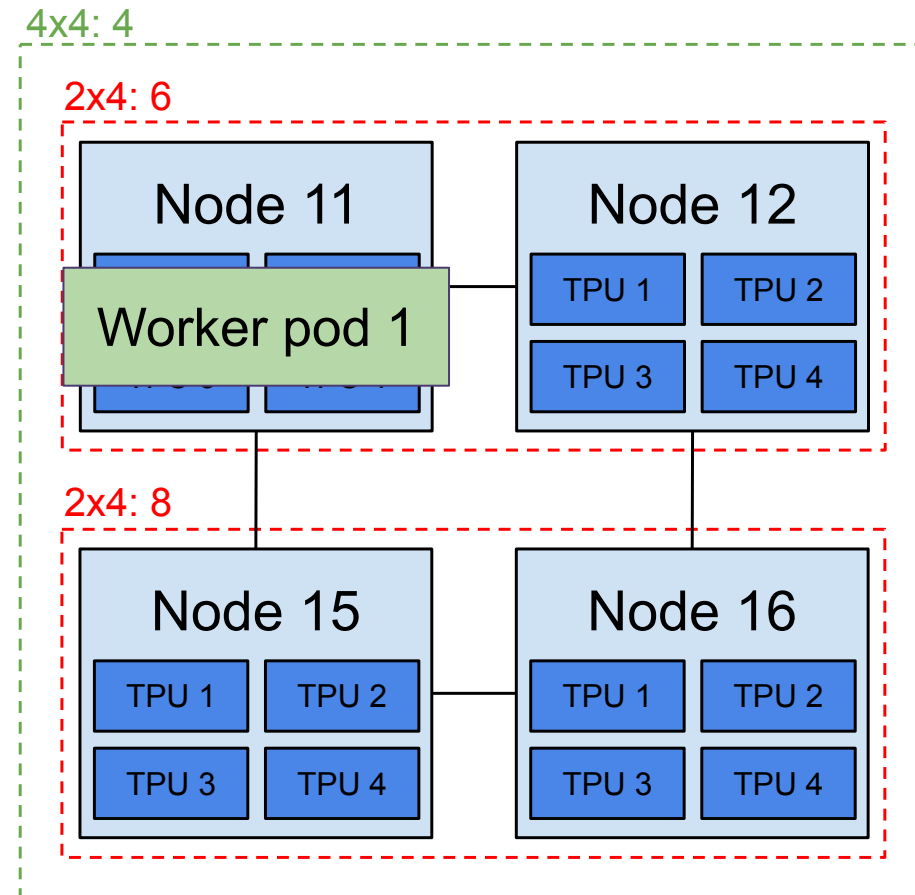
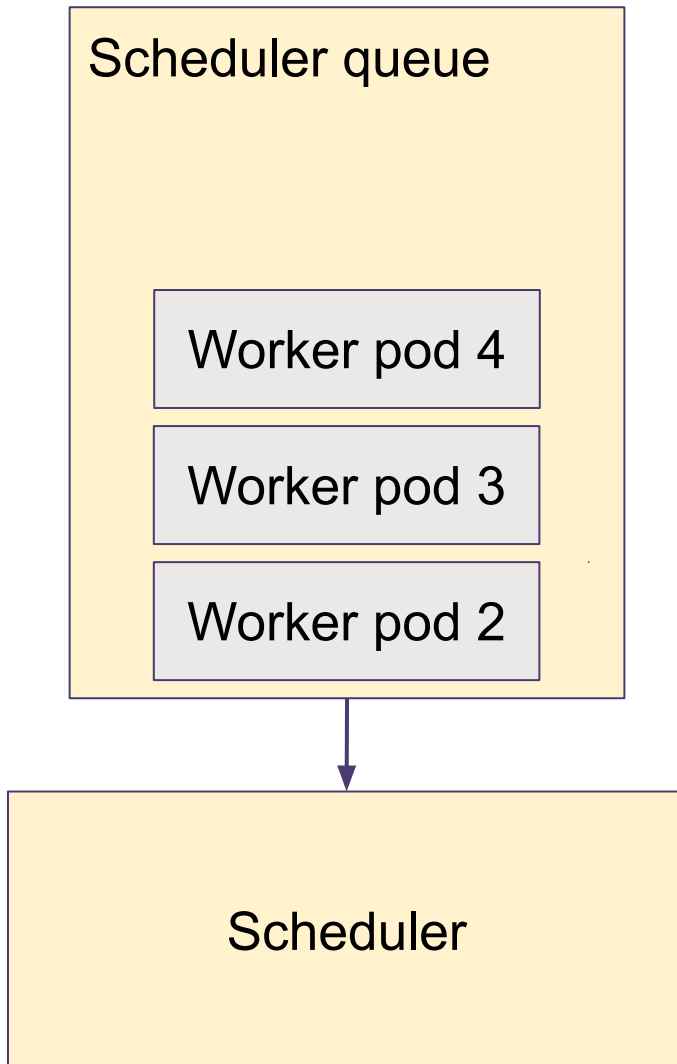


KubeCon

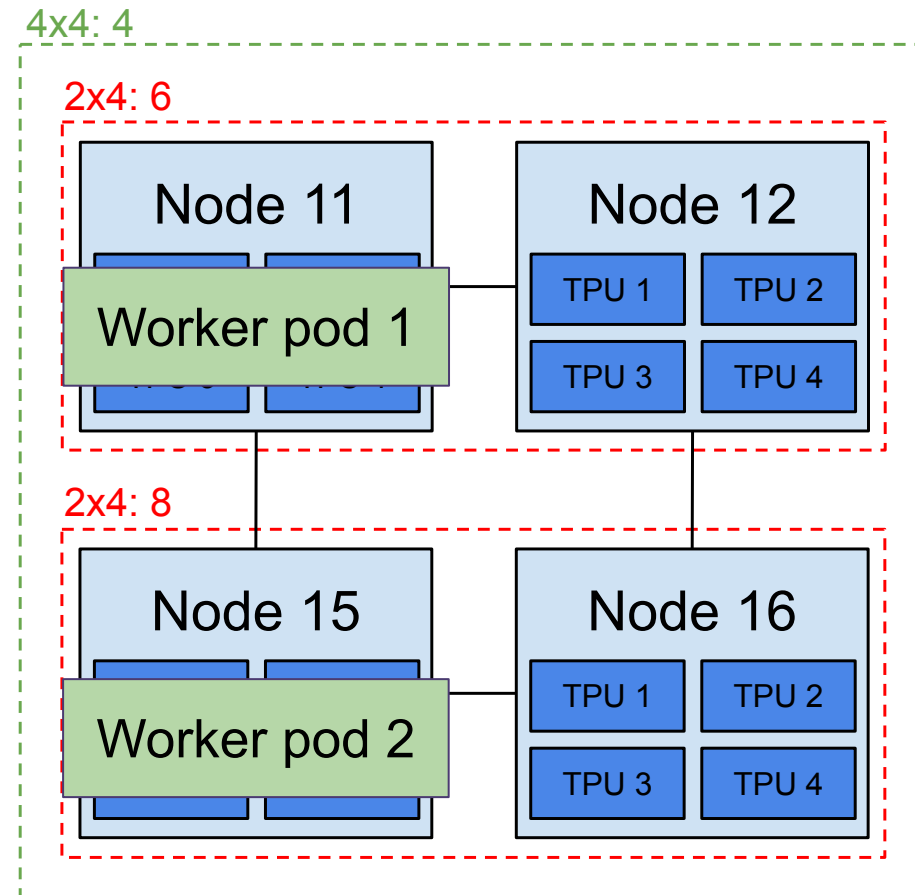
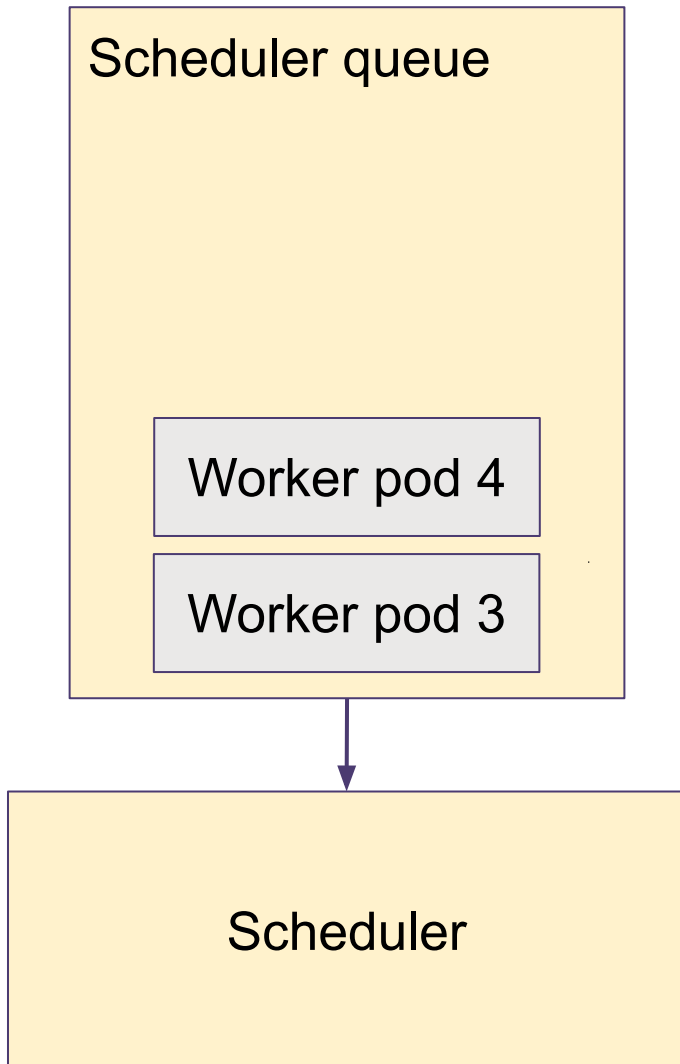


CloudNativeCon

Europe 2025



Multi-host TPUs without DRA



Multi-host TPUs without DRA

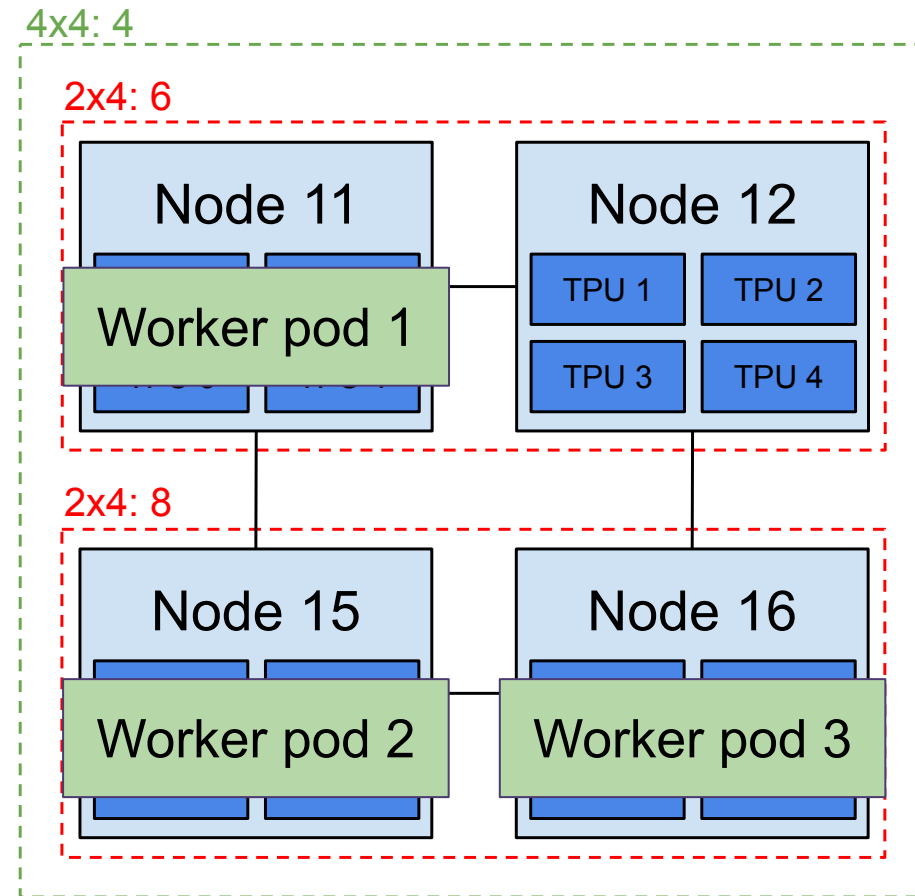
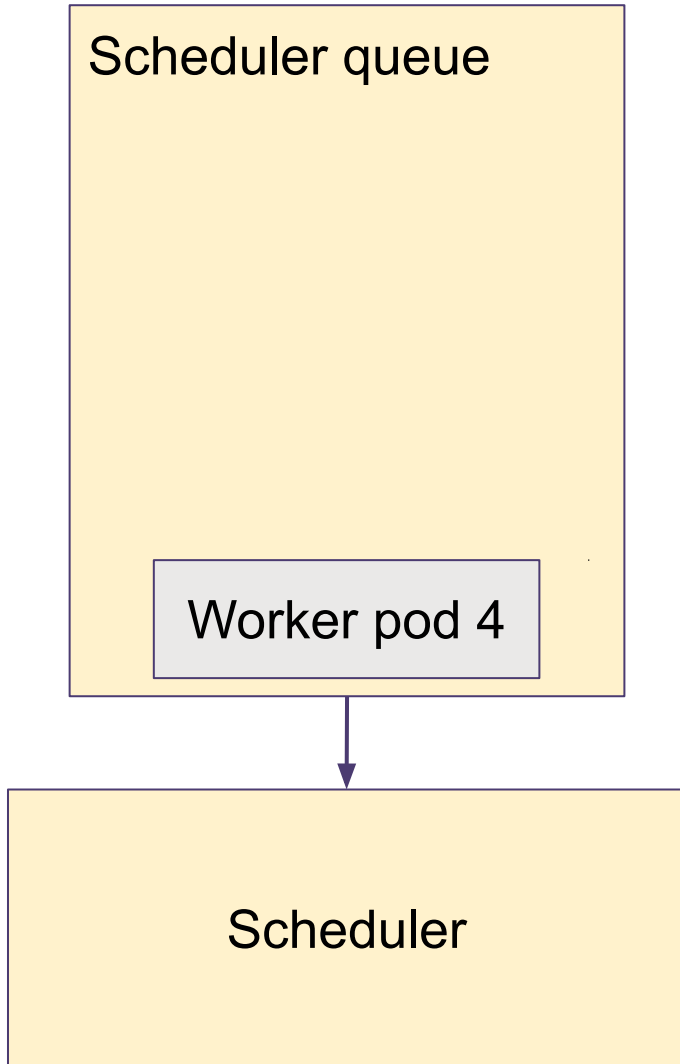


KubeCon

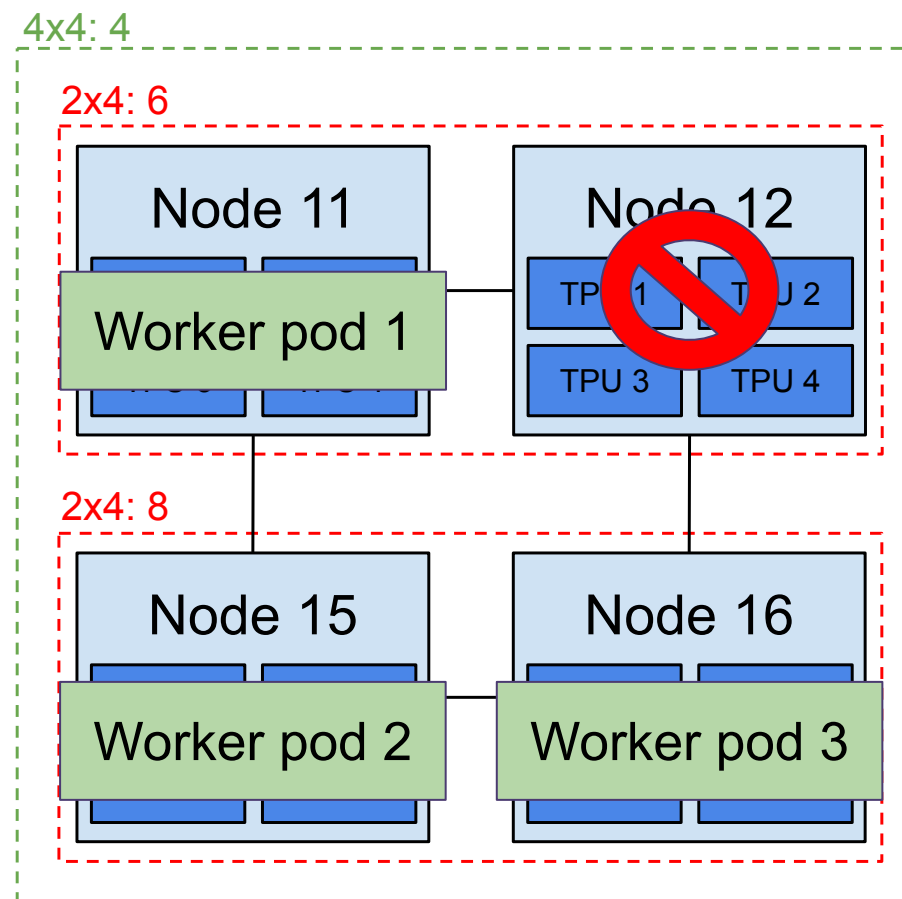
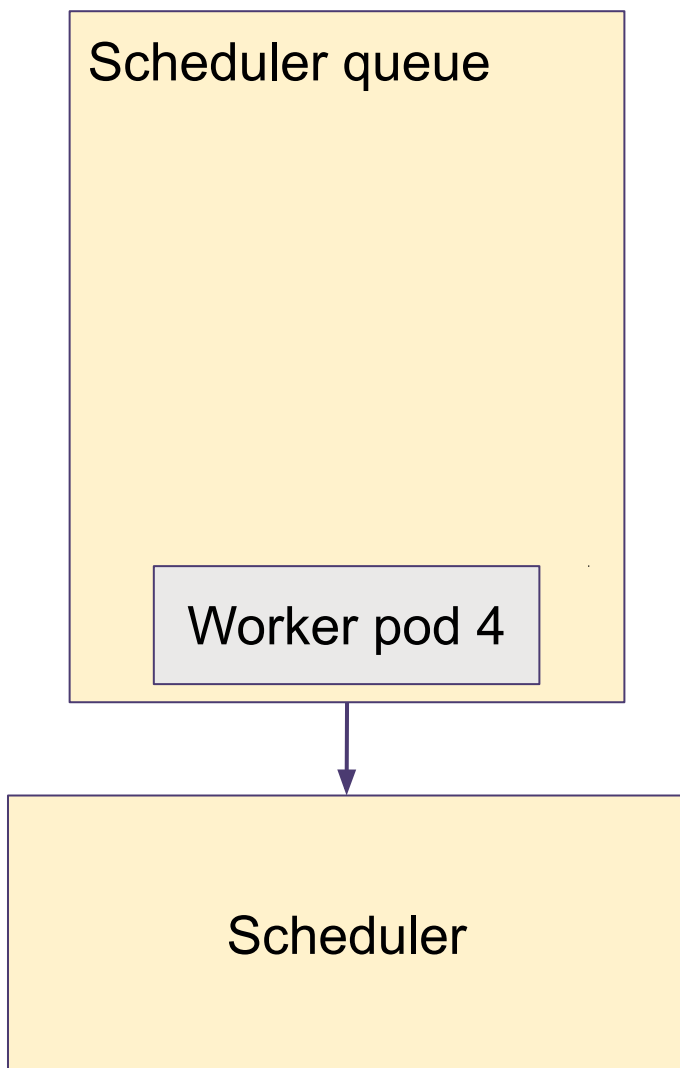


CloudNativeCon

Europe 2025



Multi-host TPUs without DRA



Multi-host TPUs without DRA

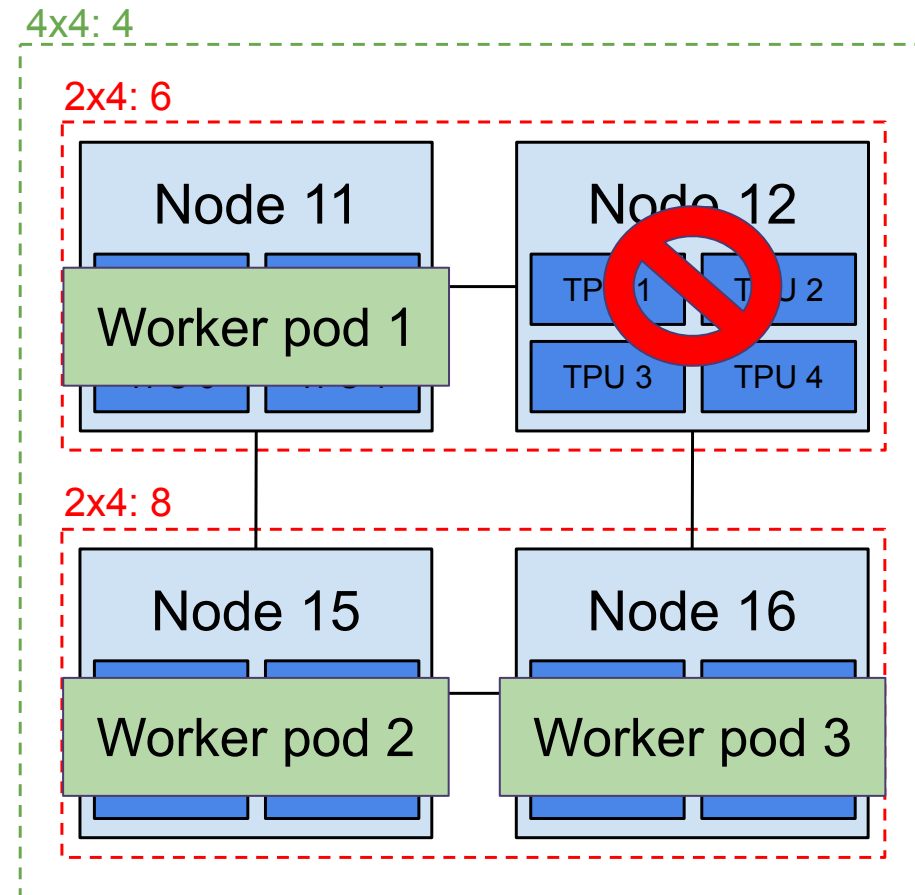
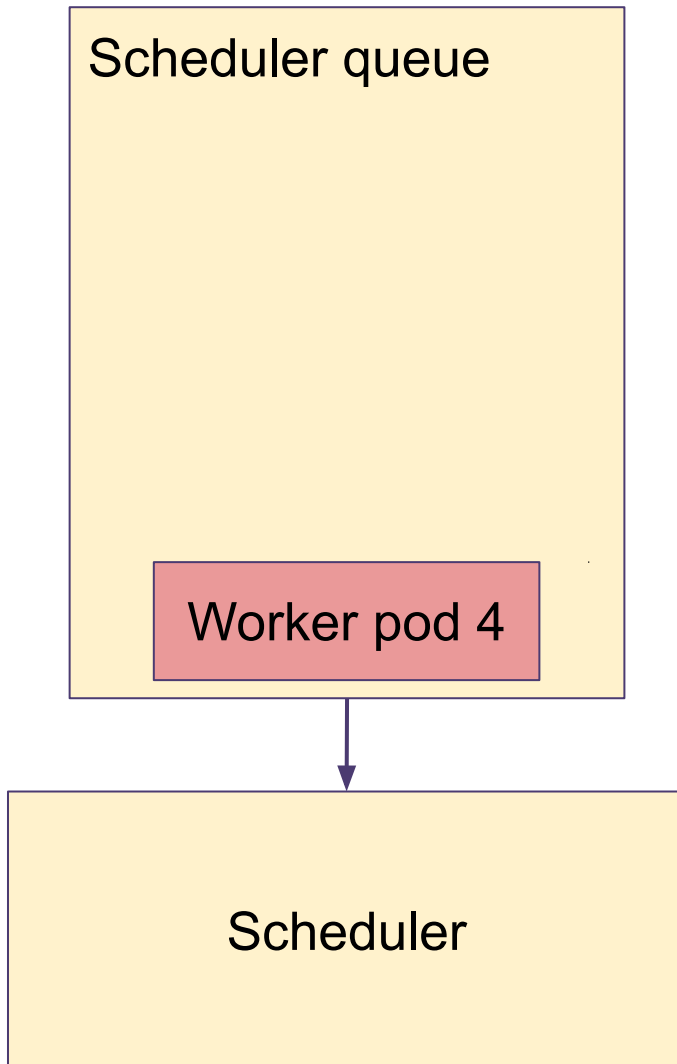


KubeCon



CloudNativeCon

Europe 2025



- First 3 worker pods on the 3 healthy nodes.
- Last node failed, worker pod can't be scheduled
 - Workload can't run
- Pods are pinned to specific nodes
 - Scheduler can't move the workloads to different nodes.
- You need to back it out and use a different nodeSelector

What should we do?

- Over the top solutions like Kueue can help
- DRA provides a scheduling level solution
 - New API for requesting resources
 - Alpha since 1.26
 - Major update in 1.31
 - Beta in 1.32
 - Several new features coming in 1.33
 - GA planned for 1.34

What is DRA:

- Part 1: API to **describe** devices

This device is an nvidia.com/gpu, its product ID is A100-SXM4-40GB, it has 40Gi of memory, and 3456 FP64 cores.

- Part 2: API to **request** devices

I need an nvidia.com/gpu with at least 30Gi of memory and at least 3000 FP64 cores.

- Part 3: Scheduler plugin that can **match** requests to devices
- Part 4: Kubelet API to **actuate** the schedulers decision

DRA Crash Course

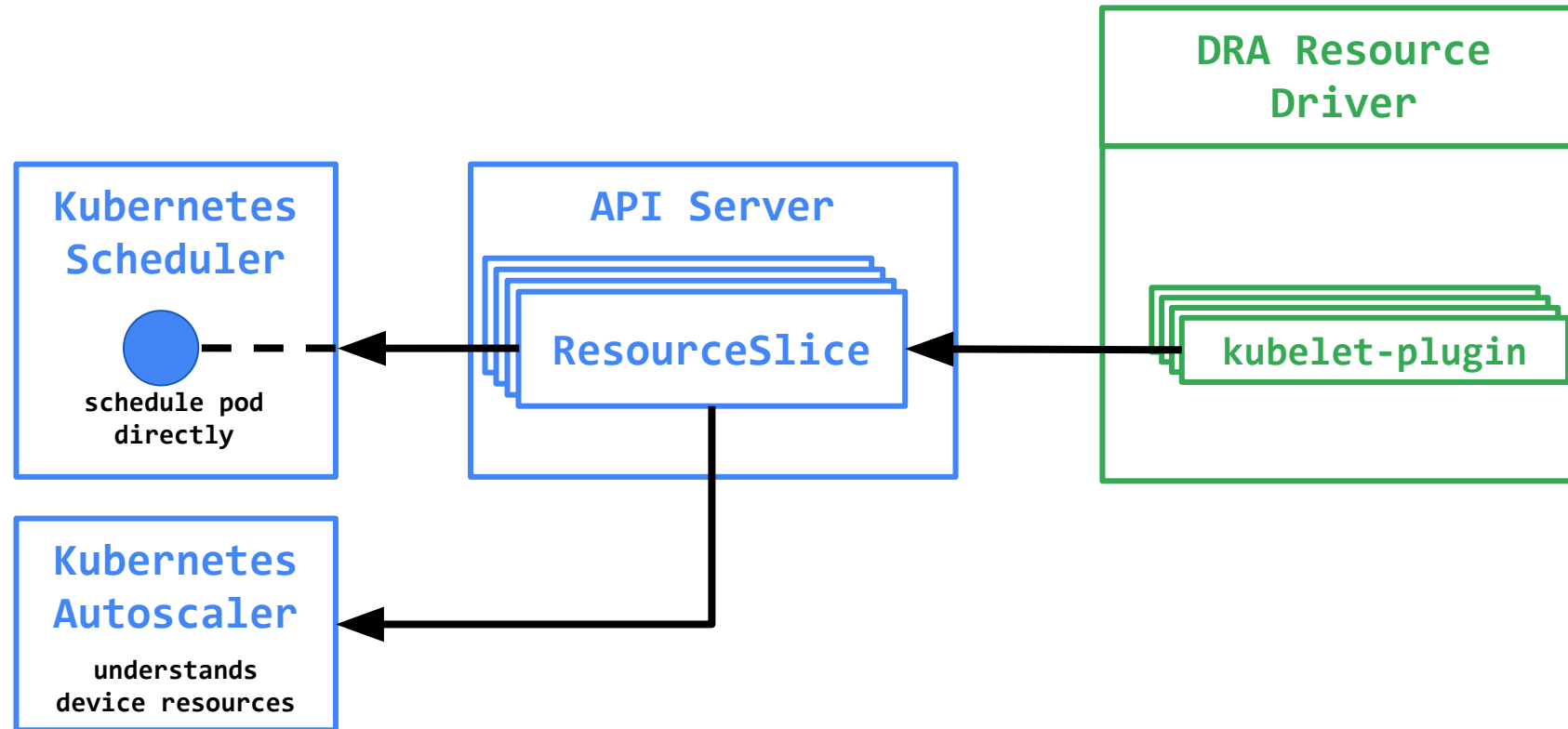


KubeCon



CloudNativeCon

Europe 2025



DRA Crash Course

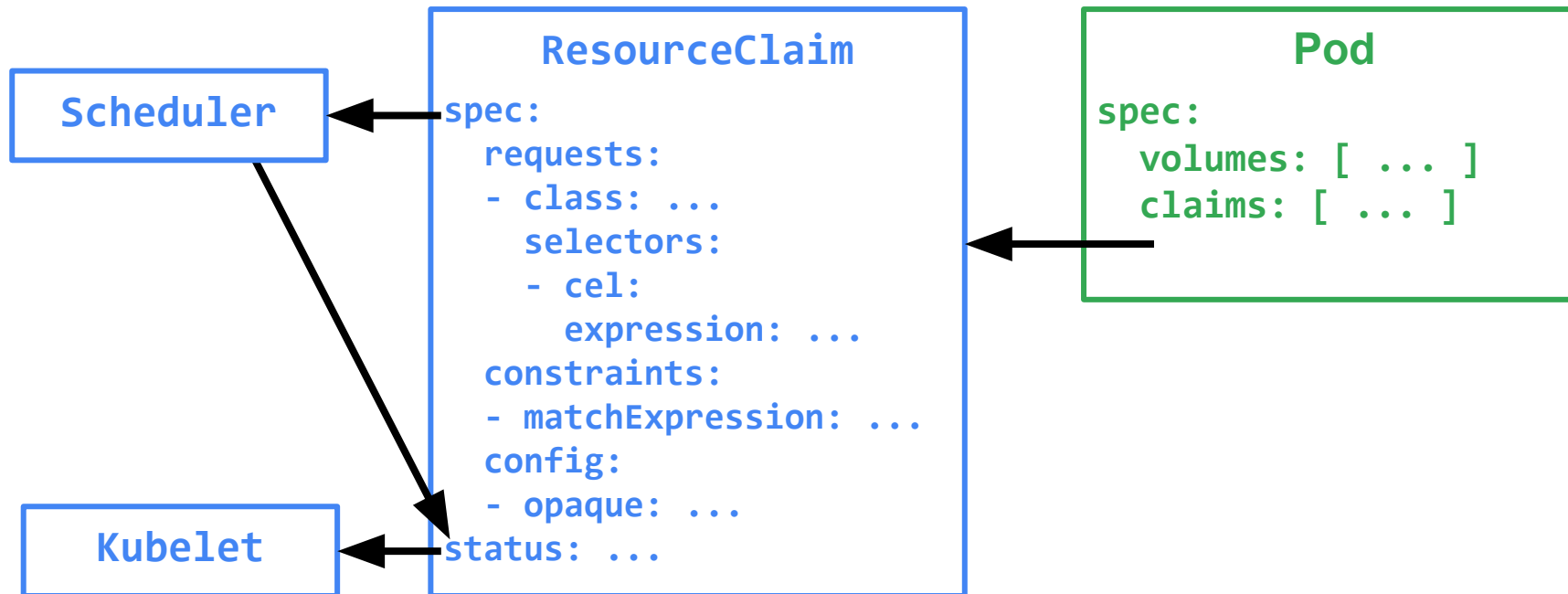


KubeCon



CloudNativeCon

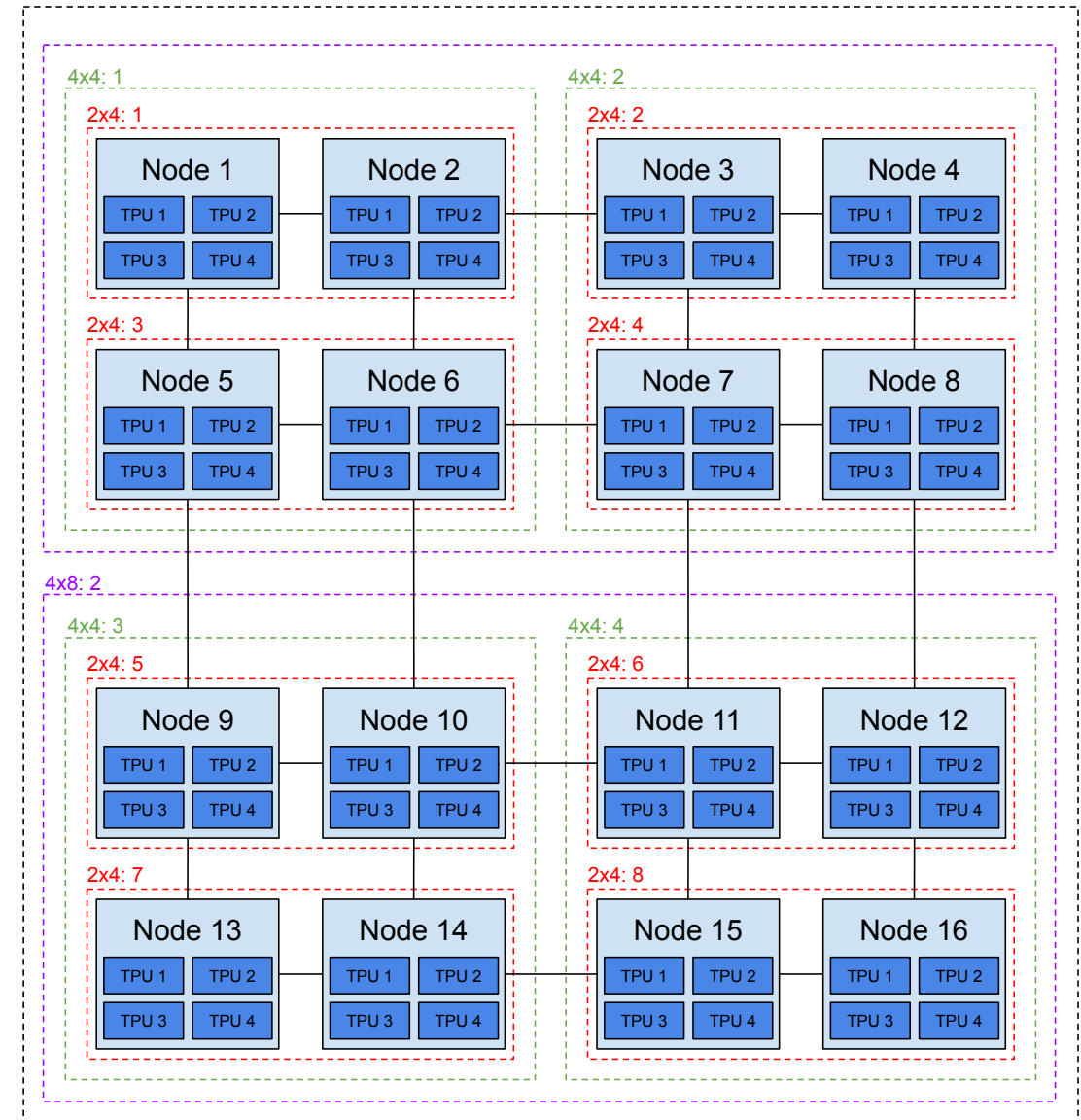
Europe 2025



- Simplest use-case: devices attached to individual nodes
- DRA can also model devices accessible from multiple nodes
- 1.33 adds a couple useful features
 - Model devices that can be partitioned into smaller, overlapping devices
 - Device taints allow immediate eviction of pods claiming the device
- Combining these provides a solution for multi-host accelerators

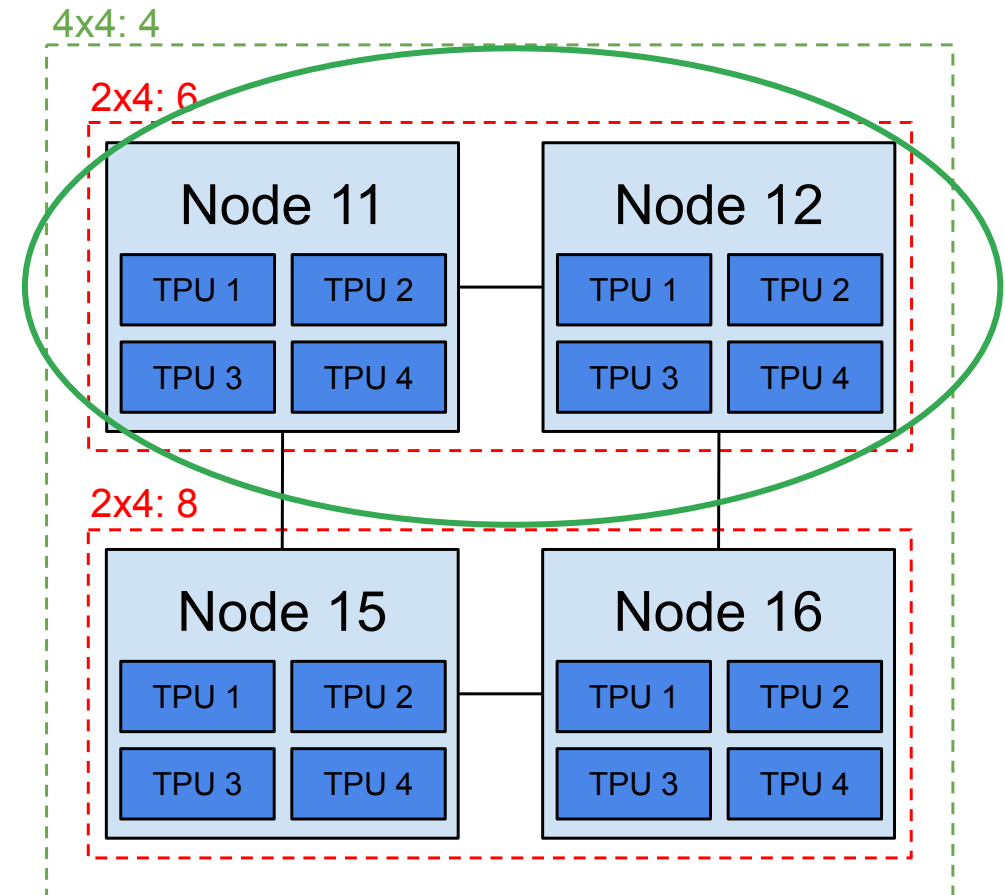
DRA modeling of multi-host devices

- 1 multi-host TPU slice = 1 logical device
 - Advertises total TPU capacity
- Node selector per logical device
 - Specifies the nodes in the TPU slice
- Nodes are in more than one TPU slice
 - DRA manages the overlap



DRA for multi-host

- DRA understands the relationships between devices.
- Allocating one device might make others devices unavailable:
 - If the first 2x4 slice is allocated, then the 4x4 and the 2x2 slices on Node 11 and Node 12 can no longer be allocated as they are using the same TPUs.



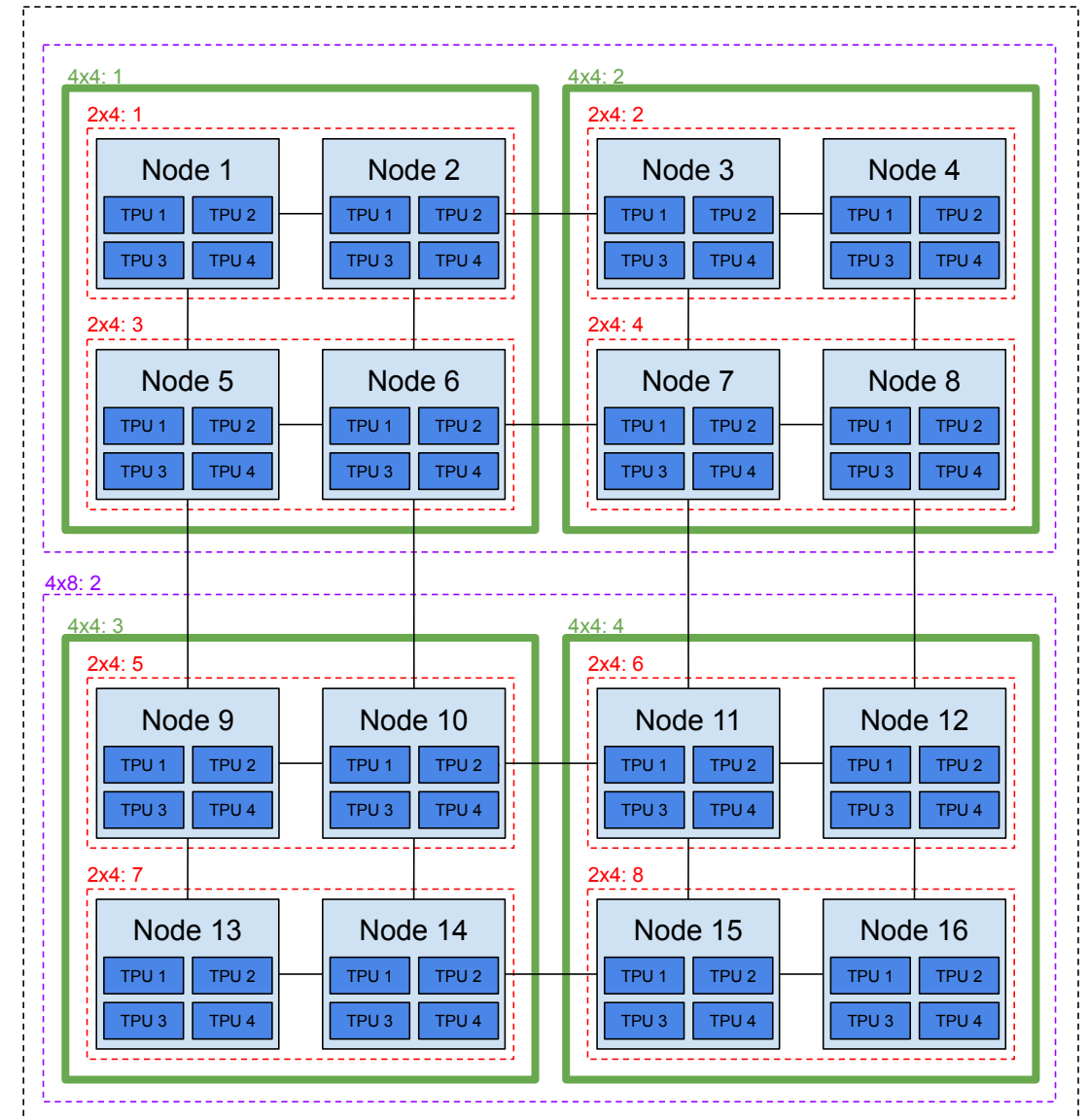
- ResourceClaim requests a device with 16 TPUs
 - 4 TPUs per node means 4 nodes
 - 4 nodes means 4 pods
 - All 4 pods point to the same ResourceClaim

```
apiVersion: resource.k8s.io/v1beta1
kind: ResourceClaim
metadata:
  name: tpu-claim
spec:
  devices:
    requests:
      - name: tpu-request
        deviceClassName: tpu.google.com
        selectors:
          - cel:
              expression: 'device.capacity["tpu.google.com"].tpus == quantity("16")'
```

```
apiVersion: v1
kind: Pod
spec:
  ...truncated...
  resourceClaims:
    - name: "tpu"
      resourceClaimName: tpu-claim
```

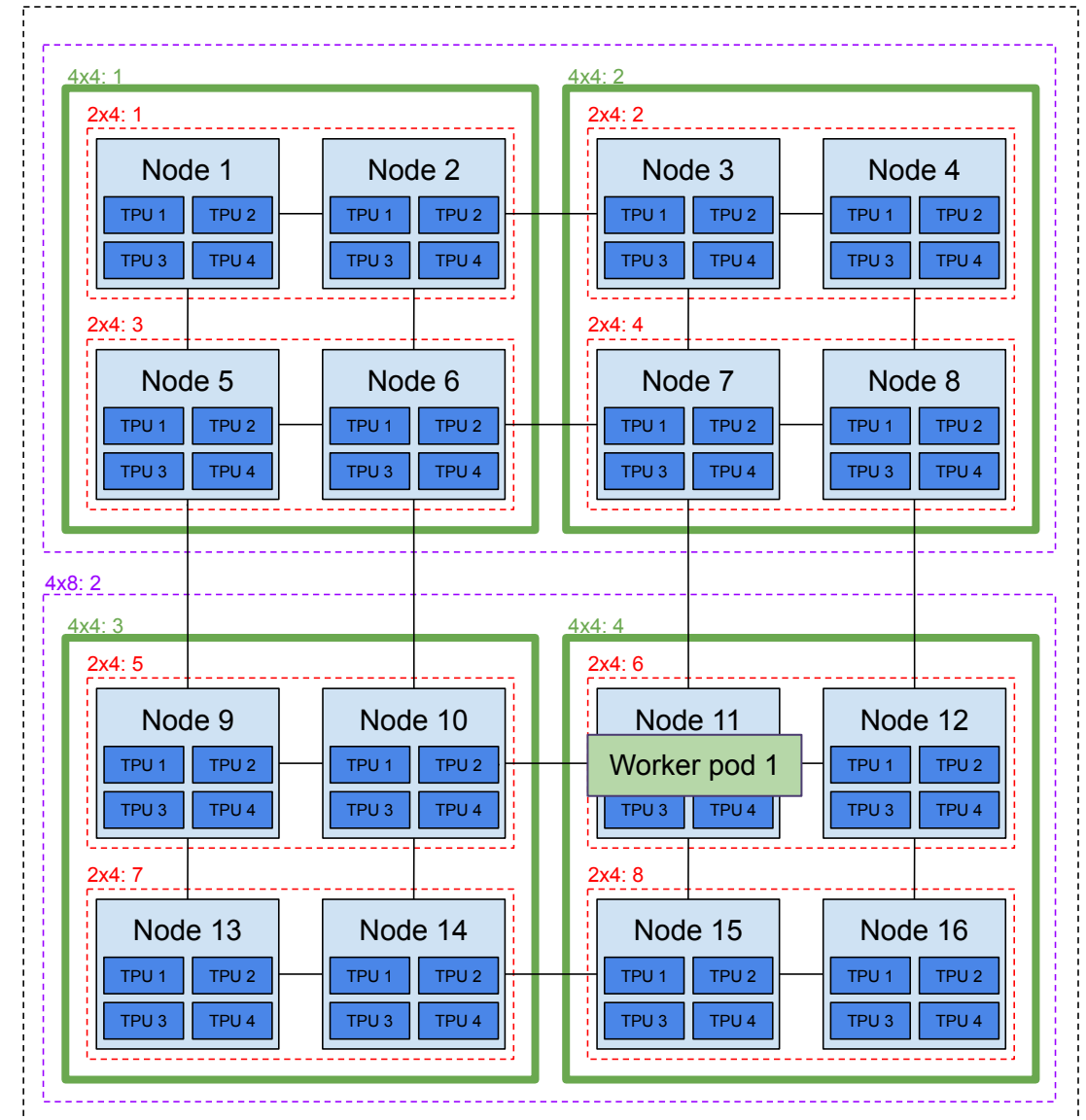
DRA for multi-host

- ResourceClaim asks for 16 TPUs
 - 4 devices that meet the criteria
- Scheduler chooses
 - Today: first fit
 - Future: Scoring?



DRA for multi-host

- Claim is allocated (device selected) when first referencing pod scheduled
- All referencing pods constrained to those nodes
- Equivalent to manually picking node selector
- But - it is dynamically chosen by scheduler



DRA for multi-host

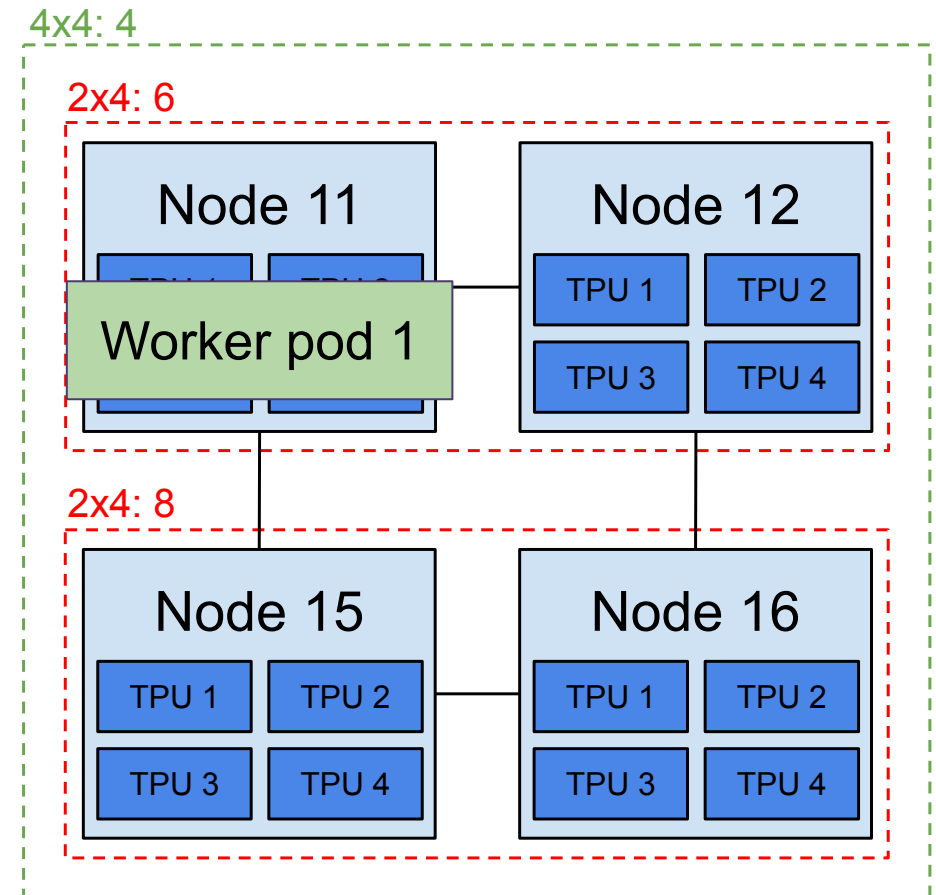
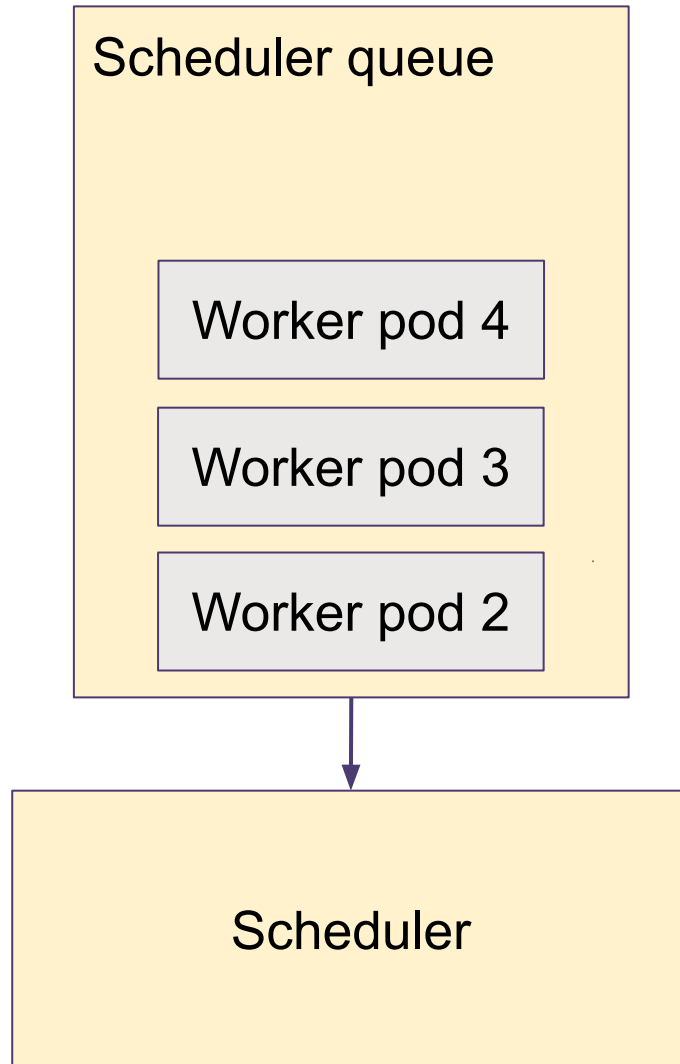


KubeCon

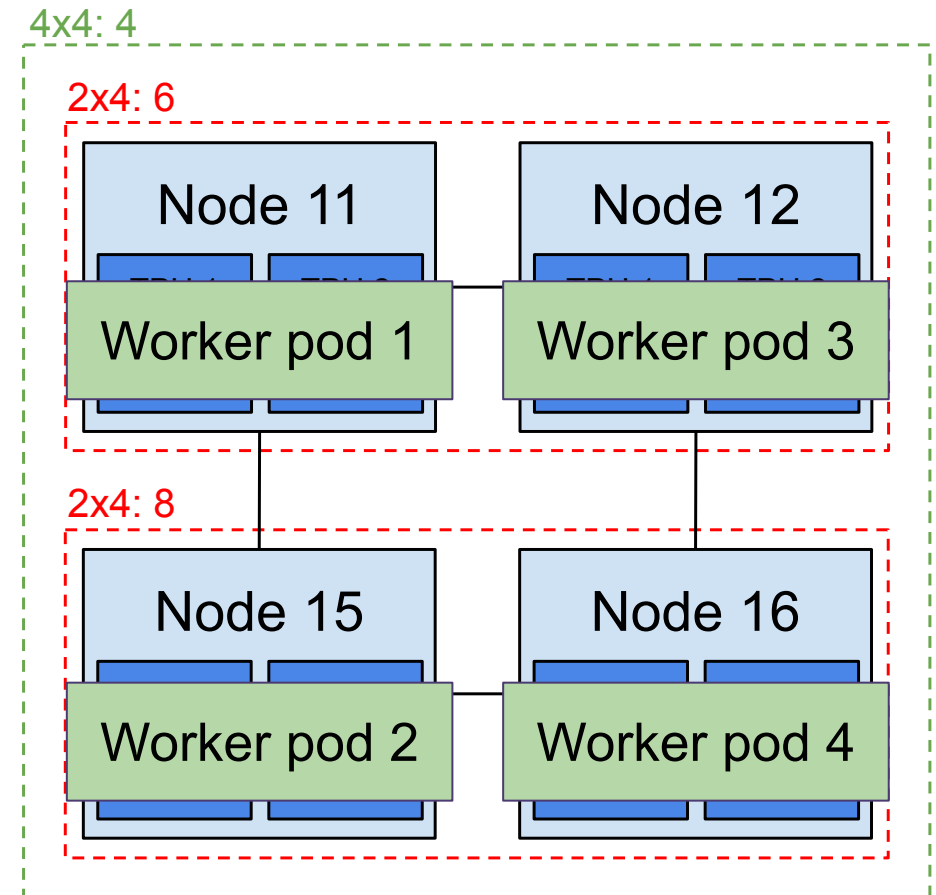
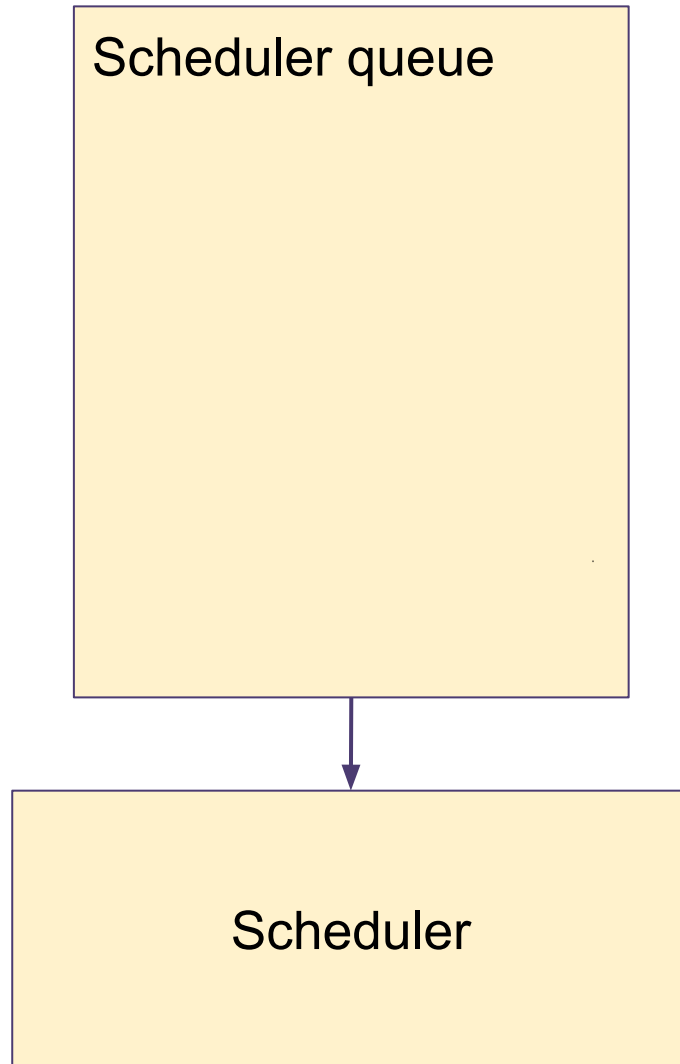


CloudNativeCon

Europe 2025

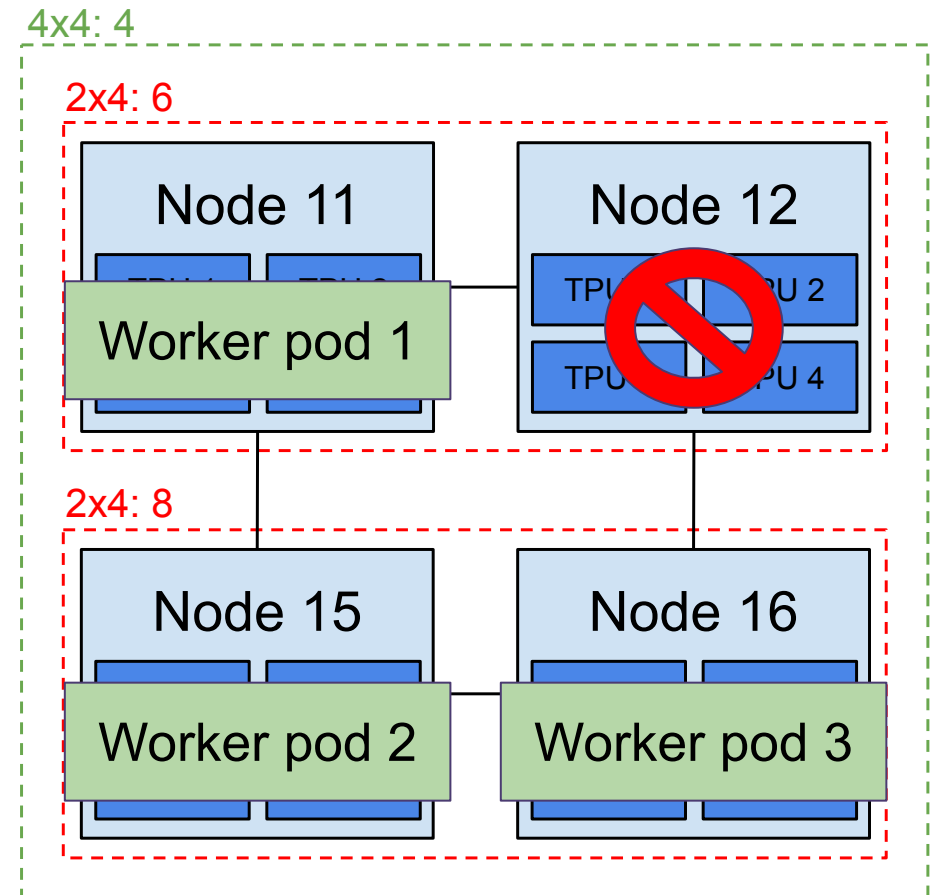
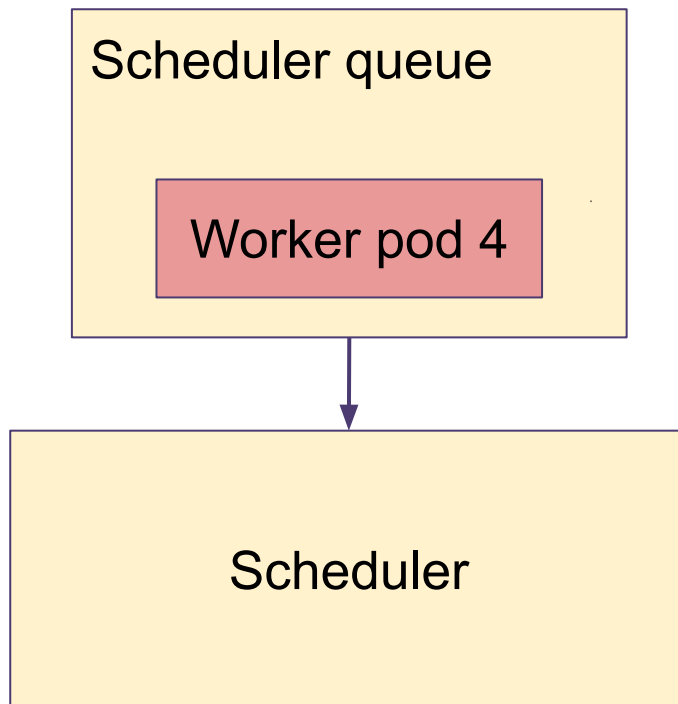


DRA for multi-host



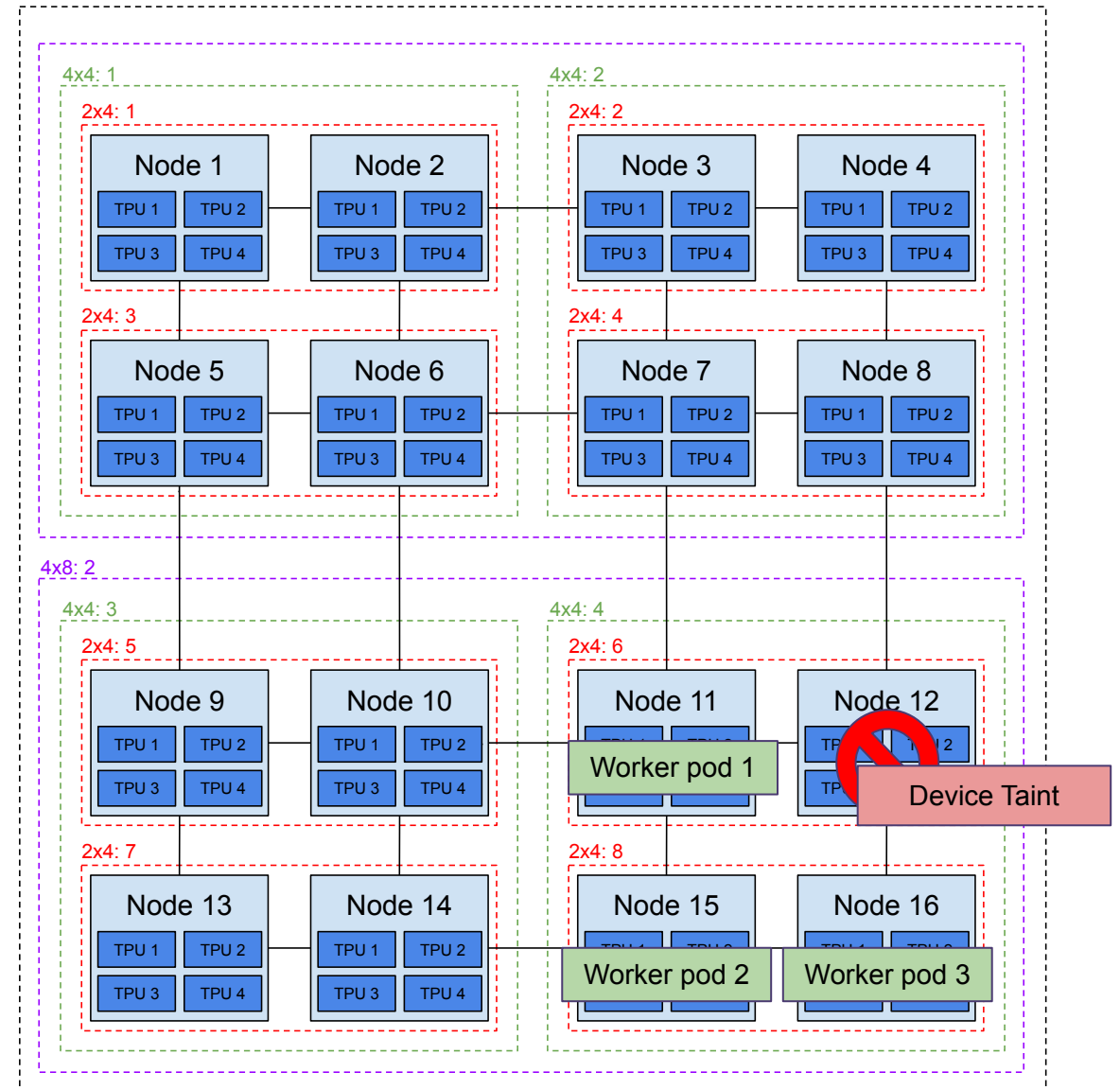
DRA for multi-host

But what if Node 12 fails?



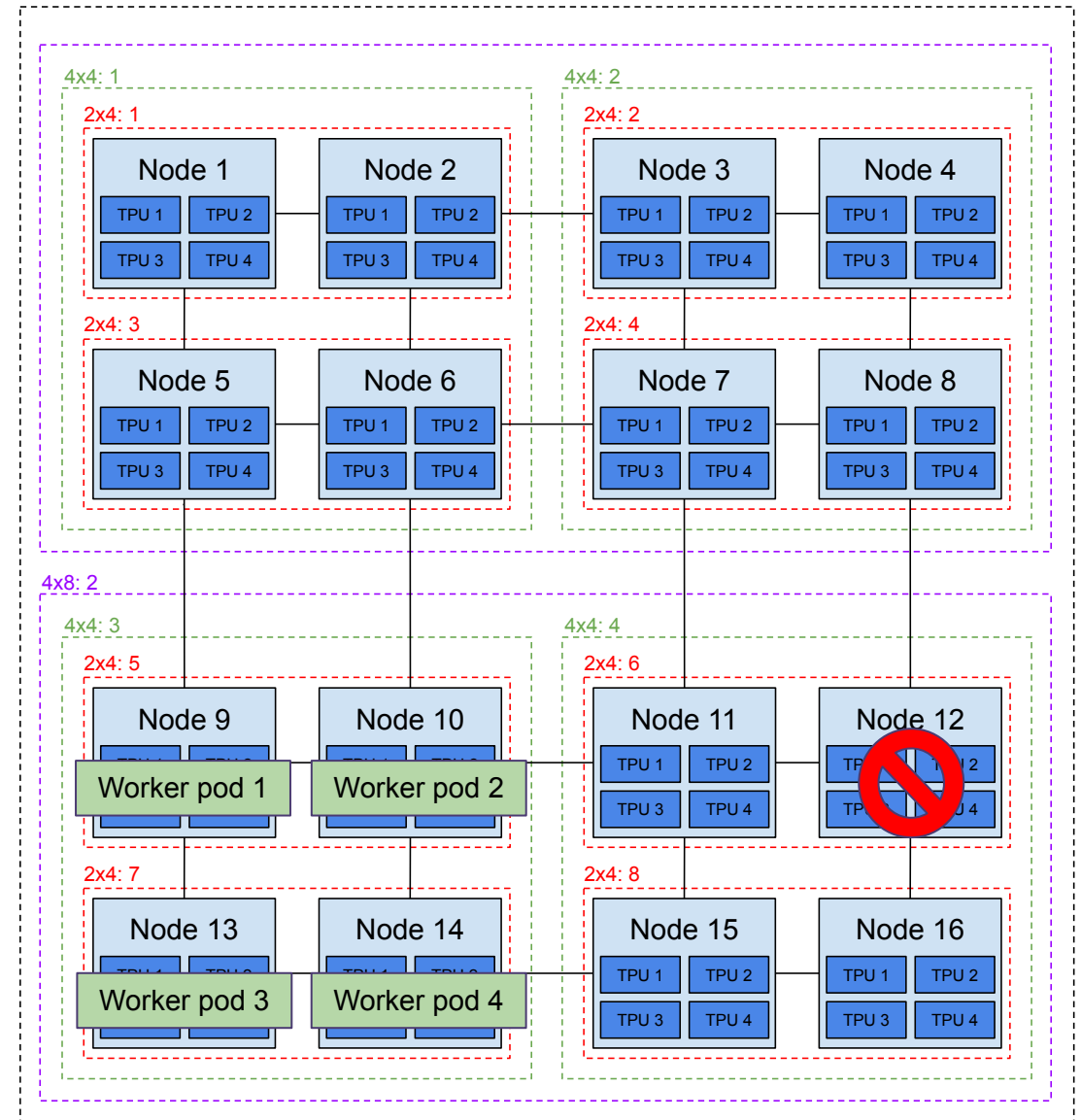
DRA for multi-host

- Node failed $\Rightarrow$ device not available
- Driver taints device with NoExecute
 - *Device*, not *Node*
- All pods claiming the device evicted



DRA for multi-host

- Pods evicted and Pending
- For first Pod, scheduler picks new 4x4 device
- All pods rescheduled to those nodes



- Pods not guaranteed to land on different nodes. Options:
 - Request enough resources that only one can fit on each node
 - Anti-affinity to force different nodes
 - Additional per pod claim for all TPUs on a node
- Scoring devices for best fit
- Supporting preemption based on workload priorities
 - High priority workload (Inference) requiring a smaller slice device should result in preemption of a low priority workload (batch) running on a larger slice

What about GPUs?



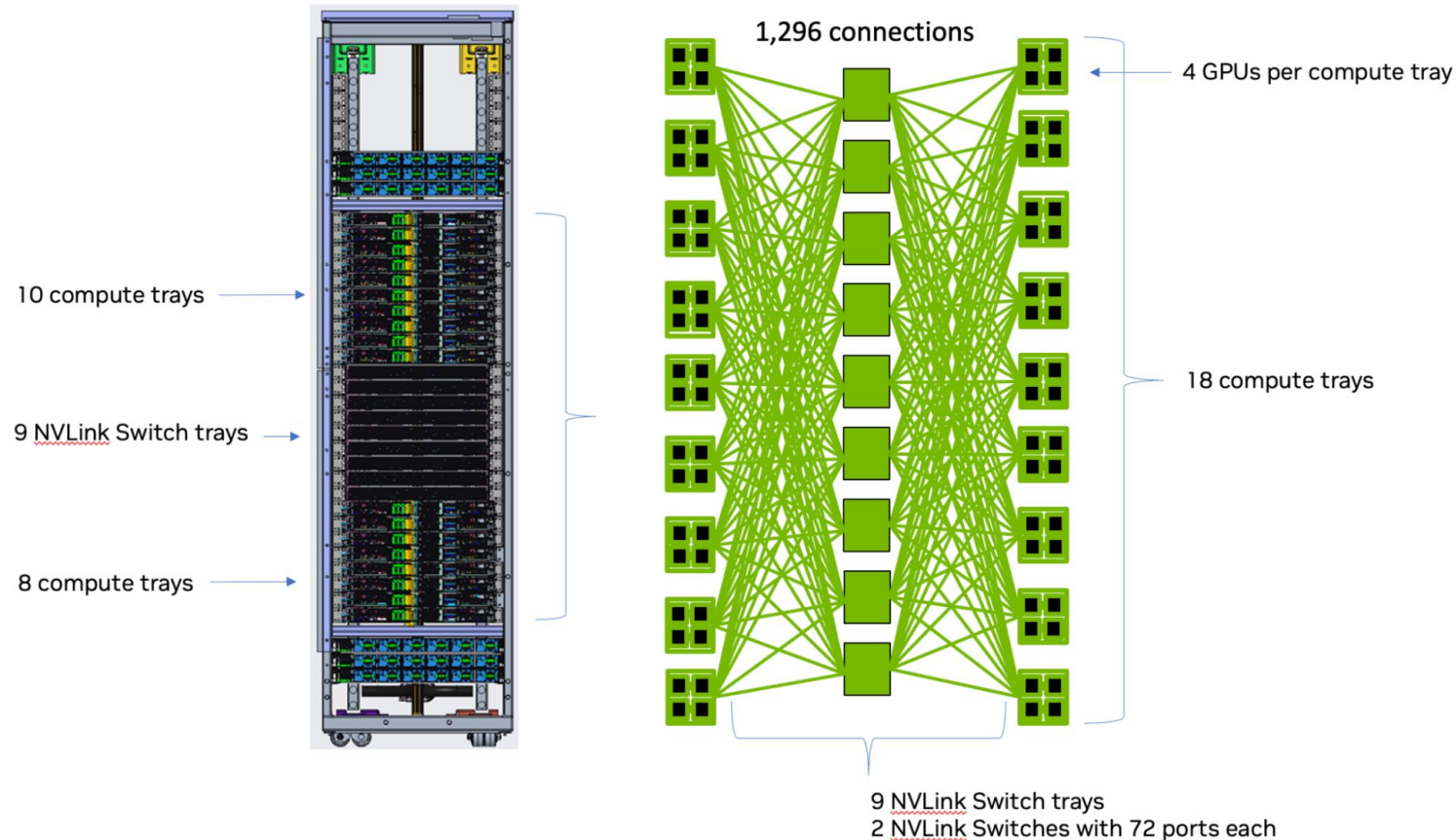
KubeCon



CloudNativeCon

Europe 2025

NVIDIA GB200 NVL72



Kubernetes WG Device Management - GPUs, TPUs, NICs and More With DRA
Kevin Klues (NVIDIA), Patrick Ohly (Intel)

Thursday, 3:00pm - 3:30pm BST | Level 3 | ICC Capital Suite 10-12

```
apiVersion: resource.nvidia.com/v1beta1
kind: ComputeDomain
metadata:
  name: compute-domain
spec:
  numNodes: 18
  channel:
    resourceClaimTemplate:
      name: compute-domain-channel
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-workload
spec:
  replicas: 18
  template:
    spec:
      containers:
        resources:
          limits:
            nvidia.com/gpu: 4
          claims:
            - name: channel
      resourceClaims:
        - name: channel
          resourceClaimTemplateName: compute-domain-channel
```

Upcoming Talks



KubeCon



CloudNativeCon

Europe 2025

- [Kubernetes WG Device Management - GPUs, TPUs, NICs and More With DRA - Kevin Klues, NVIDIA & Patrick Ohly, Intel](#)
- [More Nodes, More Problems: Solving Multi-Host GPU/TPU Scheduling With Dynamic Resource Allocation - John Belamaric & Morten Torkildsen, Google](#)
- [Project Lightning Talk: Spiderpool Updates For AI Workloads: DRA, NRI, and RDMA Observability - Weizhou Lan, Maintainer and Technical Lead](#)
- [Scaling GPU Clusters Without Melting Down! - Alay Patel & Ryan Hallisey, NVIDIA](#)
- [The GPUs on the Bus Go 'Round and 'Round - Natalie Bandel & Ryan Hallisey, NVIDIA](#)
- [Uncharted Waters: Dynamic Resource Allocation for Networking - Miguel Duarte Barroso, Red Hat & Lionel Jouin, Ericsson Software Technology](#)
- [GPU Sharing at CERN: Cutting the Cake Without Losing a Slice! - Diana Gaponcic, CERN](#)
- [Poster Session: Effortlessly Build High-Performance AI/ML Pipelines With Accelerator Chaining and K8s Native Tech - Kazuki Yamamoto, NTT & Derek Wang, Intuit](#)
- [Strengthening Auth in Kubernetes: Image Pulling, DRA Admin Access & Pod Certificates - Rita Zhang & Stanislav Láznicka, Microsoft](#)
- [AI, CERN, and the Quest for GPU Custody: How CERN Leverages DRA for Efficient GPU Sharing - Diana Gaponcic, CERN & Jan-Philip Gehrcke, NVIDIA](#)

Feedback and questions

